

Fair Diversity Maximization with Few Representatives

Florian Adriaens
florian.adriaens@helsinki.fi
University of Helsinki, HIIT
Helsinki, Finland

Nikolaj Tatti
nikolaj.tatti@helsinki.fi
University of Helsinki, HIIT
Helsinki, Finland

Abstract

Diversity maximization problem is a well-studied problem where the goal is to find k diverse items. Fair diversity maximization aims to select a diverse subset of k items from a large dataset, while requiring that each group of items be well represented in the output. More formally, given a set of items with labels, our goal is to find k items that maximize the minimum pairwise distance in the set, while maintaining that each label is represented within some budget. In many cases, one is only interested in selecting a handful (say a constant) number of items from each group. In such scenario we show that a randomized algorithm based on padded decompositions improves the state-of-the-art approximation ratio to $\sqrt{\log(m)/(3m)}$, where m is the number of labels. The algorithms work in several stages: (i) a preprocessing pruning which ensures that points with the same label are far away from each other, (ii) a decomposition phase, where points are randomly placed in clusters such that there is a feasible solution with maximum one point per cluster and that any feasible solution will be diverse, (iii) assignment phase, where clusters are assigned to labels, and a representative point with the corresponding label is selected from each cluster. We experimentally verify the effectiveness of our algorithm on large datasets.

CCS Concepts

• **Theory of computation** → Design and analysis of algorithms.

Keywords

Fairness, Max-Min Diversification, Padded Decompositions, Randomized Algorithms

ACM Reference Format:

Florian Adriaens and Nikolaj Tatti. 2025. Fair Diversity Maximization with Few Representatives. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD '25)*, August 3–7, 2025, Toronto, ON, Canada. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/3711896.3736950>

KDD Availability Link:

The source code of this paper has been made publicly available at <https://doi.org/10.5281/zenodo.15533799>.

1 Introduction

Diversity maximization is an extremely well-studied task with applications in data summarization [13, 16, 45], feature selection [44], facility location [19, 27, 36, 37], recommendation systems [1, 11, 25],



Figure 1: Selected $k = 20$ of existing nuclear power plants in US, at most one per state.

web search [8, 34, 35, 43], information exposure in social networks [31] and e-commerce [45], among others. The goal is to find a set of *diverse* items within the data, which means that the selected items should be highly dissimilar to each other.

In metric spaces this naturally translates to seeking a set with large distances between its elements. Of particular interest is the widely studied *max-min diversification* (MMD) objective function (also known as *remote-edge* or *p-dispersion*), which asks for a set S of size k that maximizes the smallest pairwise distance between distinct items in S [19, 39]. This problem is NP-complete, but admits a greedy $1/2$ approximation in metric spaces [36, 37].

Recent work has introduced *group fairness* constraints to the MMD problem [3, 29, 33, 40–42]. One important motivation is that in some data subset selection tasks, directly combining numerical and categorical attributes into a single objective function can be troublesome (although methods exist), as it might require ad hoc decisions for discretizing numerical attributes or other pre-processing [33]. Instead, formulating fairness as a constraint allows to achieve a specified level of representation of certain groups that are defined through categorical attributes, while diversity aims at achieving a high distance between numerical feature vectors.

The assumption is that the universe of items $V = V_1 \cup \dots \cup V_m$ is partitioned into m disjoint groups (also called *colors* or *labels*). A *fair* solution is a set $S \subseteq V$ of fixed size k , while requiring that S contains an appropriate amount of items from each color.

Specifically, Wang et al. [42] introduced the *fair max-min diversification* (FMMD) problem. This problem models fairness by requiring $\ell_i \leq |S \cap V_i| \leq u_i$ for each $i \in [m]$, where ℓ_i and u_i are specified parameters. These constraints capture and generalize various existing notions of group fairness such as proportional representation [13, 18] and equal representation [26]. Aside from diversity maximization, these constraints have been used in various other settings like ranking [14], clustering [26], multiwinner voting [13] and submodular maximization [18].

The FMMD problem is formally described as:



This work is licensed under a Creative Commons Attribution 4.0 International License. *KDD '25*, August 3–7, 2025, Toronto, ON, Canada
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1454-2/2025/08
<https://doi.org/10.1145/3711896.3736950>

PROBLEM 1 (FAIR MAX-MIN DIVERSIFICATION (FMMD) [42]). Assume a metric space (V, d) with distance function $d : V \times V \rightarrow \mathbb{R}_{\geq 0}$, a partition $V = V_1 \cup \dots \cup V_m$ into m colors, and parameters k, ℓ_i and u_i for each $i \in [m]$. Select a set $S \subseteq V$ of fixed size $|S| = k$, such that for all $i \in [m]$: $\ell_i \leq |S \cap V_i| \leq u_i$, while maximizing

$$\text{div}(S) = \min_{\substack{u, v \in S \\ u \neq v}} d(u, v). \quad (1)$$

Here, $\text{div}(S)$ denotes the *diversity score* of the set S .

FMMD with Few Representatives. This paper studies FMMD in case one is interested in only a *few representatives* per color.

There are plenty of applications where this assumption is well-motivated. Indeed, consider the case where one requires *at most one representative* from each color. Such fairness has been considered in prior work [1, 24, 30]. Specifically, the seminal paper of Abbassi et al. [1] considers document presentation in the context of product search or news aggregator websites. They formulated a problem where no more than one document from the same category (e.g., from the same news channel or product brand) may appear in the output. Note that also the classic MMD problem (without fairness) is a special case of FMMD with $k \leq m$, by assigning each point $v \in V$ with a unique color.

A second example is following facility location problem. Suppose the U.S. government has decided to reduce its current nuclear power plants to k plants. They have a set of current locations from which they wish to select k suitable options. Each state has agreed to the placement of *at most* one such plant within their borders. For safety reasons it is desirable that the pairwise distance between any two chosen locations is as large as possible. Where should they place the k nuclear power plants? Figure 1 shows the output of our algorithm on the current 54 plants in 28 states to $k = 20$.

A final application is the formation of committees [24]. For example, a university faculty wishes to form a committee, consisting of one member from each department in the faculty. While this may seem like an easy task, there might be faculty members who are in severe conflict with each other. Preferably they should not be on the committee together, in order to improve the decision making efficiency of the board. How to form a committee where no two members are in conflict?

Results. Our main contribution is a novel algorithm for FMMD called BREACH. We show the following guarantee when $k \leq m$:

THEOREM 1. *If $k \leq m$, then Algorithm 4 approximates FMMD within a factor of $\sqrt{\log(m)/(3m)}$ in polynomial time (see Section 3.4).*

We should point out that even though we focus on the case $k \leq m$, our algorithm can be immediately extended to the $k > m$ regime by adding $k - m$ artificial colors with $\ell_i = u_i = 0$. This comes at a cost of reducing the guarantee to $\sqrt{\log(\max(k, m))/(3 \max(k, m))}$.

Table 1 shows a summary of the known approximation ratios for the FMMD problem. The first three algorithms listed have exponential running time. Among polynomial time algorithms, our algorithm improves upon the state-of-the-art $\frac{1}{m+1}$ approximation ratio from Addanki et al. [3] by an additional $O(\sqrt{\log m})$ factor.

We should note that the existing polynomial-time algorithms [3, 33] solve a variant of FMMD with *exact size group fairness constraints*. These require exactly k_i items from each color i , and thus a

Table 1: Summary of the known approximation ratios for FMMD. A ratio of $\alpha \leq 1$ indicates the approximate solution has value at least $\alpha \times \text{OPT}$. Here ρ is the ratio of the largest and the smallest distance, $m' = \max(m, k)$, and T is the number of repetitions, affecting the success probability. The asterisk indicates the results that only hold for the regime $k \leq m$.

Paper	Approx. ratio	Run time
[42]	1	$O(n^k \log n)$
[33]	1/5	$O(kn + k^2(em)^k)$
[42]	$(1 - \epsilon)/5$	$O(mkn + m^k \log \frac{1}{\epsilon})$
[33]	$1/(3m - 1)$	$O(kn + k^2 m^2 \log(km))$
[3]	$1/((m+1)(1+\epsilon))$	$O(nkm^3 \epsilon^{-1} \log n)$
ours*	$\sqrt{\log m}/(3m(1+\epsilon))$	$O(Tm^2 n^2 \epsilon^{-1} \log \rho)$
ours*	$\sqrt{\log m}/(5m(1+\epsilon))$	$O((kmn + Tm^3 k^2) \epsilon^{-1} \log \rho)$
ours	$\sqrt{\log m'}/(3m'(1+\epsilon))$	$O(Tm' m n^2 \epsilon^{-1} \log \rho)$
ours	$\sqrt{\log m'}/(5m'(1+\epsilon))$	$O((kmn + Tm' m^2 k^2) \epsilon^{-1} \log \rho)$

special case of Problem 1 with $\ell_i = u_i$ for each color i , whereas we propose a polynomial-time algorithm that works with the general case.

Both algorithms [3, 33] use a similar approach that first builds well-separated clusters of nearby points, followed by an assignment phase to find at most one point from each cluster such that the fairness constraints are satisfied. However, their assignment phase only works for exact size group constraints.

It turns out that those algorithms can also be used for FMMD (with same approximation guarantees), by replacing the assignment phase with a more intricate flow procedure described in Section 3.3, which handles the FMMD constraints.

Techniques. Our algorithm BREACH uses a similar high-level approach as [3] and [33]. The idea is to construct well-separated clusters that are far apart, from which we select at most one point per cluster. Moumoulidou et al. [33] construct a graph where each connected component corresponds to such a cluster. Our approach also constructs a graph, but we further decompose each connected component by using Calinescu-Karloff-Rabani (CKR) random partitions [10]. From the obtained decomposition, we remove all *guard* vertices (certain boundary points) which ensures the final clusters are disconnected and thus diverse. We bound the probability that a fair solution is present in the obtained decomposition. Our flow procedure from Section 3.3 will find such a feasible solution, if it exists. By repeating the algorithm enough times, we find a solution with good probability.

2 Related Work

MMD and MSD. The most fundamental problem settings for diversity maximization assume the input is given by a metric space (V, d) with nonnegative distance function $d : V \times V \rightarrow \mathbb{R}_{\geq 0}$ and an integer k . The goal is to select a set $S \subseteq V$ of fixed size $|S| = k$ which maximizes a certain objective function.

The two most popular objective functions are *max-sum diversification* (MSD) and *max-min diversification* (MMD). MSD maximizes the sum of the pairwise distances in S (also known as *remote-clique*),

while MMD maximizes the minimum pairwise distances in S (also known as *remote-edge* or *p-dispersion*). It has been argued that the MMD objective is more suited for tasks like summarization and feature selection, because MSD has a tendency to select outliers and include highly similar items, whereas MMD covers the data range more uniformly [33, 42]. In this work, we focus on MMD.

The classic MMD problem asks for a set of k items under the max-min objective, without any fairness constraints. Both MMD and MSD are shown to be NP-complete by [19] and [39]. Tamir [37] and Ravi et al. [36] showed that the greedy “next-furthest-point-strategy” yields a $\frac{1}{2}$ -approximation for MMD, which is the well-known GMM algorithm. The GMM algorithm is identical to the Gonzalez-heuristic [22] which gives a 2-approximation for the k -center problem. Ravi et al. [36] showed that for any $\epsilon > 0$ MMD is NP-hard to approximate within a factor of $\frac{1}{2} + \epsilon$, thereby showing the optimality of the Gonzalez-heuristic in terms of worst-case performance. Additionally, Ravi et al. [36] showed that MMD without the triangle inequality constraints is NP-hard to approximate within any multiplicative factor. Amagata [5] consider MMD in the presence of outliers. Kumpulainen et al. [28] have studied the MMD problem with asymmetric distances between the points, while still retaining the directed triangle inequality. They provided a $\frac{1}{6k}$ approximation for this asymmetric generalization. MMD has also been studied in the dynamic setting [6, 17].

Fair MSD. Fairness for the MSD objective is well-understood and admits a $1/2 - \epsilon$ guarantee via local search algorithms in polynomial time [1, 9]. This is in contrast with the fair MMD problem formulations, which appear to be more difficult to approximate. Attaining a ratio larger than $1/2$ for fair MSD is hard under the planted clique hypothesis [7]. Most authors provide results for general (partition) matroids constraints [1, 7, 9, 12], which includes the fairness case when exactly k_i items are required from each color [3, 33, 40, 41]. Cevallos et al. [15] extend the setting to distances of negative type and also use local search to design a $O(1 - 1/k)$ approximation.

Fair MMD. The first to incorporate fairness constraints into the MMD problem was Moumoulidou et al. [33]. They study a variant of FMMD which considers exact size group fairness constraints (see Section 1). Moumoulidou et al. [33] provided a $\frac{1}{4}$ approximation for the case of $m = 2$ colors, a $\frac{1}{5}$ approximation for the case where $m = O(1)$ and $\sum_i k_i = o(\log n)$, and a $\frac{1}{3m-1}$ approximation for the general case. This was subsequently improved to the current state-of-the-art $\frac{1}{m+1}$ approximation guarantee by Addanki et al. [3]. The authors also showed a $\frac{1}{2}$ approximation using an LP-based randomized rounding method, but which only satisfies the fairness constraints in expectation. Using concentration bounds—which requires that each k_i is large enough—the same randomized rounding method can be used to obtain a $\frac{1}{6}$ approximation while guaranteeing that $(1 - \epsilon)k_i$ points are selected from each color.

Specifically for the case for Euclidean metrics, Addanki et al. [3] proved that the problem can be solved exactly in one dimension. For constant dimensions, they give a $1 + \epsilon$ approximation that runs in time $O(nk) + 2^{O(k)}$ where $k = \sum_i k_i$. They showed how to reduce the running time to a polynomial in k at the expense of only picking $(1 - \epsilon)k_i$ points from each color. The running time was recently improved by Kurkure et al. [29] to linear in n and k using the multiplicative weight update on an associated linear program in

combination with geometric data structures. The exact-size FMMD variant has also been studied in the context of streaming and sliding window models [3, 40, 41].

The (general) FMMD problem (Problem 1) was introduced by Wang et al. [42]. They propose an exact algorithm with runtime $O(n^k \log n)$, by reducing it to the problem of finding a maximum independent set in an undirected graph (subject to fairness constraints), which is solved by an integer linear program. Additionally, using coresets constructions they obtain a $\frac{1-\epsilon}{5}$ -approximation that runs in exponential time. They empirically show their algorithm scales to large datasets when m and k are not too large.

We should note that Moumoulidou et al. [33] also argued that their algorithm extends to (partition) matroid constraints. However, since the FMMD fairness constraints do not have matroid structure, the algorithms by Addanki et al. [3] and Moumoulidou et al. [33] are not directly applicable to the FMMD problem.

Interestingly, the best lower bounds for approximating these fair MMD problem formulations are still identical to bounds for unconstrained MMD. There remains a large gap between the polynomial time approximation ratios from Table 1 and the $\frac{1}{2} + \epsilon$ lower bound.

Independent Transversals. Theorem 1 gives an approximation guarantee for FMMD when $k \leq m$. This assumption includes cases where one requires at most a constant number of items from each color. Such fairness is closely related to finding an *independent transversal* (IT) in a graph.¹ Finding an IT is known to be NP-complete [4, 21, 24]. The IT problem has many natural applications where one is interested in selecting a unique representative from each group (the *transversal* requirement), while asking that the representatives are in some sense far apart (the *independence* requirement) [24]. Our paper studies the optimization variant of the IT problem in metric spaces with a max-min objective.

3 Algorithm

Our algorithm consists of 3 phases: (i) Given a universe V we select a subset U of V such that U still has an approximate solution and any two points in U of the same color are far away from each other. (ii) We randomly partition U in clusters such that the points of a feasible solution, with a certain probability, will be in their own clusters and the points from different clusters are far away. (iii) We use maximum flow to assign clusters to colors, and extract points of those colors from the clusters. It follows that, if the second step was successful, the resulting set will satisfy the constraints.

We present two variants of our algorithm: a slower version with a better guarantee and a faster version with a worse guarantee. The difference between the two algorithms is the preprocessing step. We assume that we know the optimal score OPT. In Section 3.4 we use grid search to find an (approximate) guess for OPT.

3.1 Preprocessing step by pruning

We preprocess (V, d) by a simple clustering step described in Algorithm 1. The algorithm has two parameters, a distance threshold γ and a budget b .

PREPROCESS(V, d, γ, b) iterates over the colors. For each color i , the algorithm iteratively selects points and removes all points that

¹ Given a graph and a partition $V_1 \cup \dots \cup V_m$ of its vertices, an IT is an independent set of vertices $v_1, \dots, v_m$ such that $v_i \in V_i$ for each i .

Algorithm 1 $\text{PREPROCESS}(V, d, \gamma, b)$, finds a subset of V ensuring same color points are far apart while still having an approximate solution

Input: Space $(V_1 \cup \dots \cup V_m, d)$, parameter $\gamma > 0$, and a budget b .

- 1: **for** each color i **do**
- 2: Label all $u \in V_i$ as unmarked.
- 3: Set $U_i \leftarrow \emptyset$.
- 4: **while** there is unmarked point in V_i and $|U_i| \leq b$ **do**
- 5: Pick any such unmarked v .
- 6: Mark all vertices in $\{u \in V_i : d(u, v) < \gamma\}$.
- 7: $U_i \leftarrow U_i \cup v$.

Output: $U = U_1 \cup \dots \cup U_m$.

are within distance of γ of the selected ones. This is repeated until no points remain or we have selected b points of that color.

An immediate consequence of the algorithm is that the points of the same color in U will be far away from each other.

LEMMA 1. Assume (V, d) , a parameter γ , and a budget b . Let $U = \text{PREPROCESS}(V, d, \gamma, b)$. Then any points $u, v \in U$ with the same color have $d(u, v) \geq \gamma$.

This reduction comes at a cost of the optimal solution. More specifically, if we do not set budget b and set $\gamma = \text{OPT}/3$, then the optimum value in the resulting set may reduce by a factor of three.

PROPOSITION 2. Assume (V, d) with m colors and n points, $V = V_1 \cup \dots \cup V_m$. Assume $\tau > 0$ such that there is $O \subseteq V$ satisfying the constraints with $\text{div}(O) \geq \tau$. Let $U = \text{PREPROCESS}(V, d, \tau/3, n)$. Then there is $S \subseteq U$ satisfying the constraints with $\text{div}(S) \geq \tau/3$.

PROOF. For each $o \in O$, let $f(o)$ be the closest point in U with the same color. Note that f is injective as otherwise we violate the triangle inequality. Consequently, the set $S = \{f(o) \mid o \in O\}$ satisfies the constraints. Moreover, the triangle inequality implies that $\text{div}(S) \geq \tau/3$, proving the claim. $\square$

Assuming that we can compute the distance $d(u, v)$ in unit time, the running time of $\text{PREPROCESS}(V, d, \tau/3, n)$ is in $O(n^2)$, which may be too expensive for large n .

If this is an issue we can speed up the preprocessing by only considering k points from each color. This comes with additional cost of reducing the approximation by a factor of five.

PROPOSITION 3. Assume (V, d) with m colors, $V = V_1 \cup \dots \cup V_m$. Assume $\tau > 0$ such that there is $O \subseteq V$ satisfying the constraints with $\text{div}(O) \geq \tau$. Let $U = \text{PREPROCESS}(V, d, 2\tau/5, k)$. Then there is $S \subseteq U$ satisfying the constraints with $\text{div}(S) \geq \tau/5$.

The proof is an adaptation of the proof for Theorem 6 in [33].

PROOF. We say that color i is critical if $|U_i| < k$. For $o \in O$ with critical color define $f(o)$ to be the closest point to o in U with the same color. Let

$$S = \{f(o) \mid o \in O, o \text{ has critical color}\}.$$

The triangle inequality now implies $\text{div}(S) \geq \tau/5$.

Let c_i be the number of points of color i in O .

Process non-critical colors as follows. Let j be a non-critical color. Delete points from U_j that are within $\tau/5$ from any point in

Algorithm 2 $\text{DECOMPOSE}(U, d, \gamma)$, padded decomposition of space U .

Input: Space $(U_1 \cup \dots \cup U_m, d)$ with n points, parameter γ .

- 1: $\alpha \leftarrow \sqrt{\log m/m}$.
- 2: $G \leftarrow (U, E)$ with $E = \{(u, v) \in U \times U \mid d(u, v) < \gamma\alpha\}$.
- 3: $\Delta_1 \leftarrow \max(\lfloor 1/(4\alpha) \rfloor, 1)$.
- 4: $\Delta_2 \leftarrow \lfloor 1/(2\alpha) \rfloor$.
- 5: $\pi \leftarrow$ a uniformly random permutation of U .
- 6: $R \leftarrow$ a uniformly random value in $\{\Delta_1, \Delta_1 + 1, \dots, \Delta_2\}$.
- 7: $Z \leftarrow \emptyset$
- 8: **for** $j = 1, \dots, n$ **do**
- 9: $C_j \leftarrow B_G(\pi(j), R) \setminus Z$.
- 10: $Z \leftarrow Z \cup C_j$.
- 11: $D_j \leftarrow \{u \in C_j \mid d_G(\pi(j), u) < R\}$.

Output: $\{D_1, \dots, D_n\}$.

S . Since any two points in U_j are least $2\tau/5$ far away, we delete at most $|S| \leq k - c_j$ points. Therefore, there are at least c_j points left in U_j . Add c_j points to S . Performing this step for each non-critical color (in any order), will result in S that has the same color counts than O , and $\text{div}(S) \geq \tau/5$. $\square$

Assuming that we can compute the distance $d(u, v)$ in unit time, the running time of $\text{PREPROCESS}(V, d, 2\tau/5, k)$ is in $O(nkm)$. The resulting set contains $O(km)$ points.

3.2 Random decomposition of the pruned space

Our next goal is to decompose U into clusters that are far enough from each other and that a feasible solution O will have at most one point in each cluster.

Assume that we have guessed the score of the optimal solution in U , say γ . Now let $\alpha = \sqrt{\log(m)}/m$, and define a graph $G = (U, E)$, consisting of the points and edges (u, v) for which $d(u, v) < \gamma\alpha$.

Denote $d_G(u, v)$ to be the shortest path distance in G between two points u, v . Let

$$B_G(u, R) = \{v \in U \mid d_G(u, v) \leq R\}$$

be the points in G with shortest path distance of at most R from u .

Let $\pi : [n] \rightarrow U$ be a random permutation of U . Let us define $\Delta_1 = \max(\lfloor 1/(4\alpha) \rfloor, 1)$ and $\Delta_2 = \lfloor 1/(2\alpha) \rfloor$. It is easy to verify that $\Delta_1 \leq \Delta_2$ for $m \geq 1$. Select R uniformly between Δ_1 and Δ_2 .

To define the decomposition we first partition U into clusters C_j for $j \in [n]$. Each $C_j \subseteq B_G(\pi(j), R)$ consists of those points in $B_G(\pi(j), R)$ that have not been included in any $B_G(\pi(i), R)$, where $i < j$. Note that some C_j might be empty.

This step is known as a Calinescu-Karloff-Rabani (CKR) random partition of a metric space [10], with a slight parameter modification to fit our needs. CKR partitions play a pivotal role in metric embedding theory [20, 32].

Our final decomposition is $D_1, \dots, D_n$, where D_j contains vertices in C_j that have $d_G(\pi(j), u) < R$. The vertices in $C_j \setminus D_j$ are referred to as *guard* vertices and are removed from the graph.

The algorithm is given in Algorithm 2.

The following proposition states that with a certain probability a fair solution can be found in clusters $D_1, \dots, D_n$.

PROPOSITION 4. Assume (U, d) with m colors, $U = U_1 \cup \dots \cup U_m$. Assume $\gamma > 0$ such that there is $O \subseteq U$ with $\text{div}(O) \geq \gamma$. Moreover, assume that for any $u, v \in U_i$, it holds that $d(u, v) \geq \gamma$. If $k \leq m$, then with probability of $\Omega(1/m)$, $\text{DECOMPOSE}(U, d, \gamma)$ yields $\{D_1, \dots, D_n\}$ such that $|O \cap D_i| \leq 1$ for every i and $O \subseteq \bigcup D_i$.

PROOF. Adopt the notation used in Algorithm 2. Let $u, v \in C_j$. Note by definition, the shortest path from u to v in G is at most $2\Delta_2$. Due to the triangle inequality, $d(u, v) < \gamma$. This immediately implies that u and v must have different colors, and that $|C_j \cap O| \leq 1$.

Next we bound the probability that O does not have any guards.

First note that since $\alpha \leq 1/2$ for any m , we have $\Delta_2 \geq 1$, which guarantees $\Delta_1 \leq \Delta_2$. Let $o \in O$. A known result, see, for example, Lemma 1.2 by Har-Peled and Rahul [23], states that

$$p(o \text{ is a guard}) \leq \frac{1}{\Delta_2 - \Delta_1 + 1} \log \frac{|B_G(o, \Delta_2)|}{|B_G(o, \Delta_1 - 1)|}.$$

Note that

$$\Delta_2 - \Delta_1 + 1 = \lfloor 1/(2\alpha) \rfloor - \max(\lfloor 1/(4\alpha) \rfloor, 1) + 1 \geq 1/(4\alpha).$$

Let $u, v \in B_G(o, \Delta_2)$. The shortest path from u to v in G is at most $2\Delta_2$. Due to the triangle inequality, $d(u, v) < \gamma$. Therefore, u and v must have different colors, and consequently, $|B_G(o, \Delta_2)| \leq m$.

If o is in a connected component, say W , of G with less than Δ_1 vertices, then W has no guards, and therefore o cannot be a guard. Assume $|W| \geq \Delta_1$. Then $|B_G(o, \Delta_1 - 1)| \geq \Delta_1 \geq 1/(4\alpha)$.

In summary, if we let $\omega = 4\alpha \log(4m\alpha)$, then

$$p(o \text{ is a guard}) \leq \omega.$$

Note that o being a guard depends solely on the order of nodes in $B_G(o, \Delta_2)$. Let $o, o' \in O$ be two distinct nodes. Then since $B_G(o, \Delta_2)$ and $B_G(o', \Delta_2)$ are disjoint, the event of o being a guard is independent of o' being a guard. Thus

$$p(O \text{ not having guards}) = \prod_{o \in O} p(o \text{ is not a guard}) \leq (1 - \omega)^k.$$

There is a constant M such that for any $m \geq M$, it holds that $\omega < 1$, $1 - \omega < 1/2$, and $\log m \geq 2\omega m$. For such m , we have

$$\begin{aligned} (1 - \omega)^k &\geq (1 - \omega)^m \geq \exp\left(\frac{-\omega m}{1 - \omega}\right) \\ &\geq \exp(-2\omega m) = m^{-2\omega m / \log m} \geq 1/m, \end{aligned}$$

where the first inequality uses the assumption $k \leq m$. $\square$

Next we show that the clusters $D_1, \dots, D_n$ are far away from each other.

PROPOSITION 5. Let $\{D_1, \dots, D_n\}$ be given by $\text{DECOMPOSE}(U, d, \gamma)$. Let S be any set for which $|S \cap D_i| \leq 1$ for every D_i , then $\text{div}(S) \geq \gamma\alpha$.

PROOF. Adopt the notation used in Algorithm 2. Assume a cross-edge (u, v) in G with $u \in C_i$ and $v \in C_j$ where $i < j$. Then u must be a guard as otherwise $v \in C_i$. Thus, the clusters without the guards are disconnected. Assume now that $u \in D_i$ and $v \in D_j$ where $i < j$, then $d(u, v) \geq \gamma\alpha$. $\square$

Let us now look at the computational complexity of DECOMPOSE . The bottleneck of the algorithm is computing $B_G(\pi(j), R)$. Computing B_G independently leads to a running time of $O(n^3)$. We can speed-up computing $B_G(\pi(j), R)$ by ignoring the nodes v that have shorter distance $d_G(\pi(i), v)$ than the distance $d_G(\pi(j), v)$ for

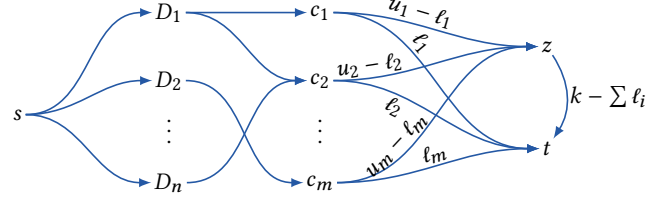


Figure 2: Flow network used to solve the assignment of clusters $\{D_i\}$ to colors. Capacities are indicated at edges. Unmarked edges have capacity of 1.

Algorithm 3 $\text{ASSIGN}(\{D_i\}, \{\ell_i\}, \{u_i\}, k)$ tries to find a feasible solution from $\{D_i\}$

Input: Clusters $D_1, \dots, D_n$, constraints $\{\ell_i\}$, $\{u_i\}$, and k .

- 1: construct flow network $H = (W, A)$.
- 2: solve the (integral) maximum flow problem for H .
- 3: $S \leftarrow \emptyset$
- 4: **for** $i = 1, \dots, n$ **do**
- 5: **if** D_i flows to c_j **then**
- 6: $u \leftarrow$ any vertex in D_i with color j .
- 7: add u to S .

Output: S

some $i < j$. Then each node is only visited $O(R)$ times, leading to computational complexity $O(Rn^2) \subseteq O(mn^2)$. In practice, a naive approach is reasonable.

3.3 Assigning colors to clusters

We will now describe our final step. In order to do this, assume that DECOMPOSE has produced a decomposition $\{D_1, \dots, D_n\}$ such that there is a solution $S \subseteq \bigcup D_i$ satisfying all the constraints and that every node in S is in its own cluster D_i , that is, $|D_i \cap S| \leq 1$.

Our final step is to extract S , or any solution satisfying the constraints, from $\{D_1, \dots, D_n\}$. We will do this by assigning (some of) the clusters $\{D_i\}$ to colors. The assignment will be done such that each cluster has a vertex of assigned color, and that there are enough assignments to satisfy all the constraints.

In order to do this, we define a flow network $H = (W, A)$. An illustration of H is given in Figure 2. The vertices W contains n nodes, each corresponding to the cluster D_i . Moreover, W contains m nodes $c_1, \dots, c_m$, corresponding to the colors. Finally, the W has additional three nodes s , z , and t .

The edges A are as follows. We connect s to each D_i with unit capacity. For each vertex u in D_i with color j , we add an edge from D_i to c_j with unit capacity. Each c_j is connected to t with capacity ℓ_j and to z with capacity $u_j - \ell_j$. Finally, z is connected to t with capacity $k - \sum \ell_i$.

The following proposition states that we can extract the solution.

PROPOSITION 6. Assume that there is a feasible set O such that $|D_i \cap O| \leq 1$ and $O \subseteq \bigcup D_i$. Then $\text{ASSIGN}(\{D_i\}, \{\ell_i\}, \{u_i\}, k)$ yields a set S satisfying the constraints.

PROOF. Consider the following flow. Assume $D_i \cap O = \{u\}$, where u has a color j . Then push flow from s to c_j via D_i . Let s_j be

Algorithm 4 BREACH($V, d, \gamma_1, \gamma_2, b, T$), main algorithm combining all three steps

Input: Space ($V = V_1 \cup \dots \cup V_m, d$), parameters γ_1, γ_2 , preprocessing budget b , constraints $\{\ell_i\}, \{u_i\}, k$, maximum repetitions T .

- 1: $(U, d) \leftarrow \text{PREPROCESS}(V, d, \gamma_1, b)$.
- 2: $S \leftarrow \emptyset$.
- 3: **repeat** $T \times m$ times or until S satisfies the constraints
- 4: $\{D_1, \dots, D_n\} \leftarrow \text{DECOMPOSE}(U, d, \gamma_2)$.
- 5: $S \leftarrow \text{ASSIGN}(\{D_i\}, \{\ell_i\}, \{k_i\}, k)$.

Output: S

the number of vertices in O with color j . Push a flow of ℓ_j from c_j to t and a flow of $s_j - \ell_j$ from c_j to z . Finally, push a flow of z from t , at maximum capacity. Since O satisfies the constraints, we see that the flow is valid. Moreover, all edges pointed to t are saturated, making this flow optimal with a throughput of k .

Let f be an optimal flow. Since the capacities are all integral, we can assume that the flow is integral. This flow must have a throughput of k , which can only happen if the edges to t are saturated. Consequently, there are at least ℓ_i , and at most u_i , clusters flowing to c_i . Moreover, there are in total k clusters flowing. Consequently, S constructed by ASSIGN satisfies the constraints. $\square$

Let us consider the computational complexity of ASSIGN. Since every point has only one color, the graph H has $O(n + m)$ edges. A deterministic algorithm by Van Den Brand et al. [38] can solve the flow in $O((n + m)^{1+o(1)} \log^2 k)$ time. In practice, a standard flow solver such as push-relabel, suffices.

3.4 Putting everything together

Our next step is to combine all three steps together with the main algorithm given by Algorithm 4. The algorithm first preprocesses the space using γ_1 as the threshold, decomposes the space using γ_2 , and tries to find a feasible solution using maximum flow. We still need to determine γ_1 and γ_2 , which we will discuss shortly.

Next we state the approximation ratio and the computational complexity of BREACH. The first set of parameters lead to a slower algorithm due to $O(n^2)$ term but yields a better guarantee.

PROPOSITION 7. Assume a space (V, d) with n points with a set O satisfying the constraints and having a diversity score $\text{div}(O) \geq \tau$. Then BREACH($V, d, \tau/3, \tau/3, n, T$) yields a solution S with $\text{div}(S) \geq \sqrt{\log m \tau} / (3m)$ with probability $1 - 2^{-\Omega(T)}$. The computational complexity of the algorithm is in $O(Tm^2 n^2)$.

PROOF. Propositions 2, 4, 5, and 6 imply the approximation guarantee. Repeating the decomposition phase (line 3 in Algorithm 4) together with Proposition 4 gives an overall success probability of

$$1 - (1 - \Omega(1/m))^T \geq 1 - 2^{-\Omega(T)}.$$

The running time of the algorithm is equal to

$$O(n^2 + Tm^2 n^2 + Tm(n + m)^{1+o(1)} \log^2 k) \in O(Tm^2 n^2),$$

proving the claim. $\square$

The second set of parameters lead to a faster algorithm but yields a worse guarantee.

PROPOSITION 8. Assume a space (V, d) with n points with a set O satisfying the constraints and having a diversity score $\text{div}(O) \geq \tau$. Then BREACH($V, d, 2\tau/5, \tau/5, k, T$) yields a solution S with $\text{div}(S) \geq \sqrt{\log m \tau} / (5m)$ with probability $1 - 2^{-\Omega(T)}$. The computational complexity of the algorithm is in $O(kmn + Tm^3 k^2)$.

PROOF. Propositions 3, 4, 5, and 6 imply the approximation guarantee. Repeating the decomposition phase together with Proposition 4 guarantees the probability. The running time of the algorithm is equal to

$$O(kmn + Tm(km)^2 + Tm(km + m)^{1+o(1)} \log^2 k) \in O(kmn + Tm^3 k^2),$$

proving the claim. $\square$

Both propositions require τ , the optimal score. A naive option would be to test all existing distances, multiplying the running time by a factor of $O(n^2)$. A faster alternative option is to do a grid search by considering candidates of form $\tau_i = \tau_0(1 + \epsilon)^i$, where τ_0 is a distance between any two points and i is an integer. We can lower bound i as we know that if τ_i is the smallest non-zero possible distance, then BREACH will find a solution. On the other hand, if τ_i yields γ_2 that is larger than the largest distance in the pruned space, then there is no solution. Let ρ be the ratio of the largest distance and the smallest non-zero distance in V . Then we need only $O(\log_{1+\epsilon} \rho) \subseteq O(\epsilon^{-1} \log \rho)$ tests. This comes at a cost to the quality of the solution by a factor of $1/(1 + \epsilon)$, proving the approximation results stated in Table 1.

The $k > m$ regime. We can extend BREACH to work with $k > m$ by adding $k - m$ artificial colors with $u_i = \ell_i = 0$. This comes yields a weaker guarantee $O(\sqrt{\log(m')}/(3m'))$, where $m' = \max(m, k)$. Moreover, the running times of the two algorithms change to $O(Tmn^2 m')$ and $O(kmn + Tm^2 k^2 m')$.

Alternatively, we can obtain the original guarantees, but with a slower algorithm. To this end, note that Proposition 4 proves a success probability of $\Omega(1/m)$ for a single repetition of line 3 in Algorithm 4, in case $k \leq m$. Assume that k is proportional to m , say $k \leq \beta m$ for some $\beta \geq 1$. Proposition 4 would then yield a success probability of $\Omega(1/m^\beta)$. Therefore, Algorithm 4 (line 3) would need $T \times m^\beta$ repetitions to ensure that Algorithm 4 succeeds with probability $1 - 2^{-\Omega(T)}$ (see Propositions 7 and 8). The running time of such algorithm is polynomial only when β remains a constant.

3.5 Practical improvements

In order to improve the quality of the solution in practice, we do the following additional steps.

During preprocessing we do not treat colors individually. Instead we always select the point of a color that we need that is furthest from already added points. This increases the preprocessing time by $O(\log n)$ due to sort.

When forming G in DECOMPOSE it turns out that the threshold $\gamma_2 \alpha$ is too small, resulting in subpar results. To combat this we vary γ_2 independently, instead of having it fixed to τ . More specifically, we vary γ_2 between $\gamma_1/2$ and γ_1/α using a similar $(1 + \epsilon)$ grid as with τ . This increases the running time of the last two steps by $O(\epsilon^{-1} \log \alpha) \subseteq O(\epsilon^{-1} \log m)$.

Finally, we found out that in practice BREACH yields good results after only a few iterations. Thus in experiments, we perform DECOMPOSE only a constant number of times.

4 Experimental evaluation

In this section, we describe our experimental evaluation. All experiments were performed on an Intel Core i5 machine at 1.6 GHz with 16 GB RAM. Our methods are implemented in Python 3.8 and will be made publicly available.

Data. For data, we use the four real-world public datasets that were previously used for evaluation by Wang et al. [42] in their work on FMMD. The dataset characteristics are listed in Table 2. Here, *dim* refers to the dimension of the numerical feature vectors. As a distance measure, we use euclidean distance. For the *CelebA* dataset, the original dimension of the feature vectors was 25 088. We used sparse random projections [2] to reduce the dimension to 10. In all datasets, labels are defined from demographic attributes such as sex, race and age. For more details on data preparation, and other statistics such as relative proportion of each label within the data, see [42, Appendix A].

Baselines. We compare our algorithm BREACH with the three most scalable methods listed in Table 1 (Section 1). The polynomial time approximation algorithm from [33] is denoted as FAIRFLOW. The approximation algorithm from [3] is denoted as GREEDYFLOW. The scalable coresets approximation based on integer linear programming from [42] is denoted as CORE. We use the publicly available Python implementations from [42] for these methods. The other methods from Table 1 are exponential time algorithms that are not scalable to large datasets [42] and are not used for evaluation.

Parameters. We use similar parameter selection as in [42]. The fairness constraints are defined according to *proportional representation* [13]. Namely, for each label $i \in [m]$ we set $\ell_i = \max(1, (1 - \alpha)k \frac{|V_i|}{|V|})$ and $u_i = \max(1, (1 + \alpha)k|V_i|/n)$ for $\alpha = 0.2$. For FAIRFLOW and GREEDYFLOW we set $k_i = \lceil k|V_i|/n \rceil$ or $k_i = \lfloor k|V_i|/n \rfloor$. All algorithms are repeated 5 times, and we show the average diversity scores and average running time in each experiment. We set $\epsilon = 0.1$ in the grid search for BREACH and GREEDYFLOW and in CORE. We used the faster version of BREACH and ran DECOMPOSE three times.

Experimental results. Figures 3–4 compare the diversity scores and the running times of BREACH, FAIRFLOW, GREEDYFLOW and CORE for increasing values of k . For this experiment in particular, we sampled 1000 items uniformly at random from the full datasets, similar to the experimental setup in [42]. Observe that BREACH always outperforms the other polynomial time algorithms FAIRFLOW and GREEDYFLOW on the diversity score, and is often faster as well. Note that this is partially explainable by tighter constraints; however in our experiments BREACH also outperformed the other polynomial time algorithms when used with the same constraints. BREACH is competitive with the exponential time algorithm CORE.

Table 3 and Table 4 show the performance on the full datasets for $k = 15$. CORE achieves the best performance, but BREACH is a close second and outperforms FAIRFLOW and GREEDYFLOW. We did not test on the *CelebA* dataset, since the full data was not present in the repository of [42]. GREEDYFLOW did not provide a solution within 24 hours on the *Census* data set.

Table 2: Real-world datasets used for evaluation.

Data [42]	labels	m	n	dim
<i>Adult</i> (S)	sex	2	48 482	6
<i>Adult</i> (R)	race	5		
<i>Adult</i> (S+R)	sex+race	10		
<i>CelebA</i> (S)	sex	2	202 599	10
<i>CelebA</i> (A)	age	2		
<i>CelebA</i> (S+A)	sex+age	4		
<i>Census</i> (S)	sex	2	2 426 116	25
<i>Census</i> (A)	age	7		
<i>Census</i> (S+A)	sex+age	14		
<i>Twitter</i> (S)	sex	3	18 836	1,024

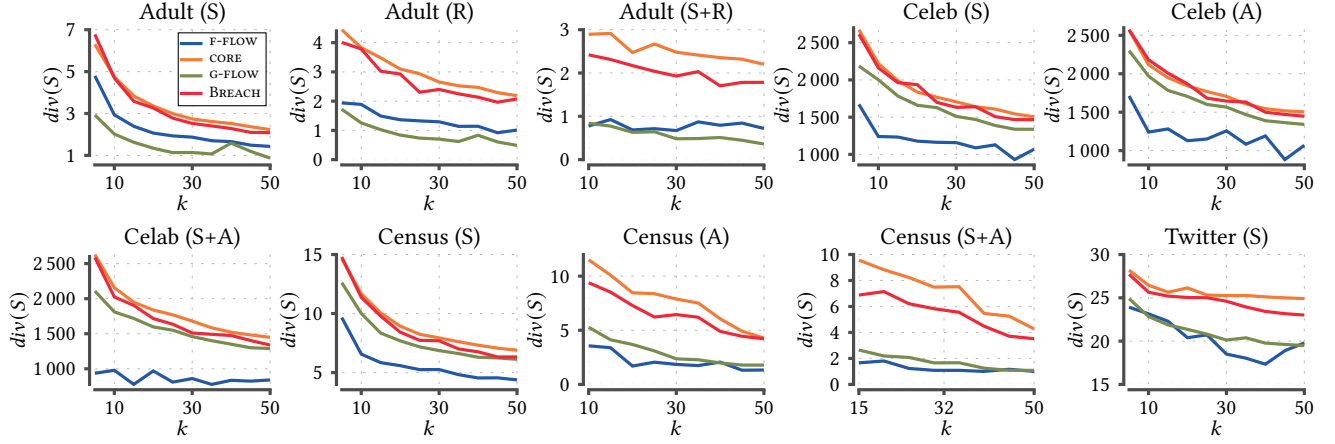
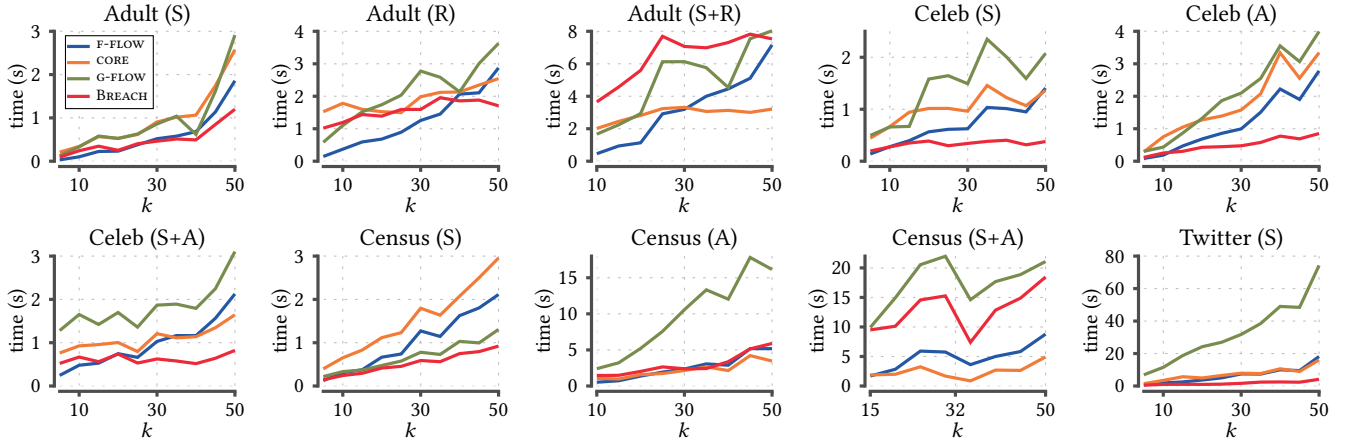
Table 3: Running time (s) of different algorithms on the full datasets from Table 2 for fixed parameter value $k = 15$.

Data	FAIRFLOW	CORE	GREEDYFLOW	BREACH
<i>Adult</i> (S)	12.88	18.07	117.93	14.93
<i>Adult</i> (R)	6.85	14.53	50.48	19
<i>Adult</i> (S+R)	11.48	21.48	44.6	45
<i>Census</i> (S)	516.88	688	-	492.46
<i>Census</i> (A)	658.6	1130.19	-	1630.8
<i>Census</i> (S+A)	892.26	843.28	-	2289.88
<i>Twitter</i> (S)	29.71	58.23	175.31	14.61

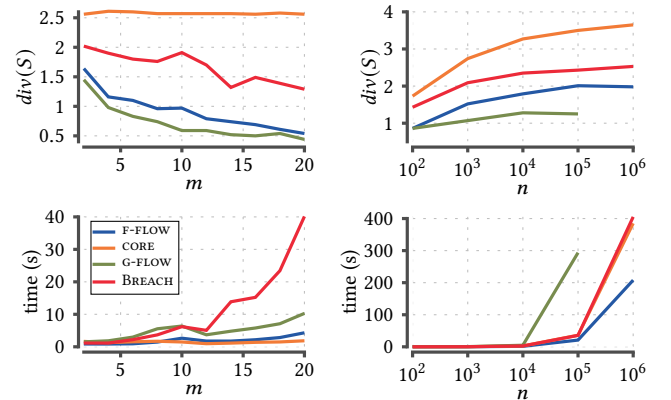
To compare the scalability of the different algorithms, we generate synthetic datasets. Using the implementation of [42] we generate ten two-dimensional Gaussian isotropic clouds with random centers in $[-10, 10]^2$ and identity covariance matrices. We set $k = 30$. First, we fix $n = 1000$ and increase m . Then, we fix $m = 3$ and increase the graph size n . Figure 5 shows the results. GREEDYFLOW did not finish within 24 hours for $n = 10^6$. BREACH scales worse for increasing m when compared to the other baselines. This is mainly due to two reasons. First, our preprocessing step is repeated for each optimal guess γ (Section 3.2), while FAIRFLOW only does a greedy preprocessing once to obtain km nodes, independent of the guess. Secondly, the CKR partitions from the decomposition phase (Section 3.2) require a BFS search for each of the km nodes. This is rather costly for increasing values of m . Nonetheless, BREACH achieves better diversity scores than the other polynomial time approximation algorithms FAIRFLOW and GREEDYFLOW, and is slightly worse than CORE. CORE is tailored for performance and scalability to large datasets (by using fast ILP solvers), but has exponential running time in the worst case.

5 Conclusions

We proposed a novel randomized approximation algorithm for FMMD called BREACH. Our algorithm uses random CKR partitions to find well-separated clusters and a novel maximum flow procedure to extract feasible solutions. We theoretically prove our algorithm outperforms existing methods in a regime of few representatives per color, and also give a guarantee for the general problem. In practice, our algorithm outperforms all other polynomial time approximation

Figure 3: Diversity scores of the solutions of different algorithms versus the solution size k on a sample of $n = 1000$.Figure 4: Running time of different algorithms versus the solution size k on a sample of $n = 1000$.Table 4: Diversity scores of different algorithms on the full datasets from Table 2 for fixed parameter value $k = 15$.

Data	FAIRFLOW	CORE	GREEDYFLOW	BREACH
Adult (S)	3.63	5.93	1.57	5.18
Adult (R)	1.67	5.49	1.13	3.96
Adult (S+R)	0.96	5.25	0.76	2.94
Census (S)	9.64	13.30	-	13.11
Census (A)	5.57	13.30	-	11.18
Census (S+A)	1.01	13.38	-	9.11
Twitter (S)	21.25	27.53	22.05	26.08

Figure 5: Diversity scores and the running times of the algorithms as a function of m and n on synthetic graphs.

algorithms, while achieving slightly worse diversity score than the best exponential time algorithm. A direction for future work is to further narrow down the gap between the $\frac{1}{2} + \epsilon$ inapproximability lower bound and the approximation ratios from Table 1.

Acknowledgments

This research is supported by the Academy of Finland project MAL-SOME (343045) and by the Helsinki Institute for Information Technology (HIIT).

References

- [1] Zeinab Abbassi, Vahab S Mirrokni, and Mayur Thakur. 2013. Diversity maximization under matroid constraints. In *Proceedings of the 19th ACM SIGKDD international conference on Knowledge discovery and data mining*. 32–40.
- [2] Dimitris Achlioptas. 2001. Database-friendly random projections. In *Proceedings of the twentieth ACM SIGMOD-SIGACT-SIGART symposium on Principles of database systems*. 274–281.
- [3] Raghavendra Addanki, Andrew McGregor, Alexandra Meliou, and Zafeiria Moumoulidou. 2022. Improved Approximation and Scalability for Fair Max-Min Diversification. In *25th International Conference on Database Theory, ICDT 2022, March 29 to April 1, 2022, Edinburgh, UK (Virtual Conference) (LIPIcs, Vol. 220)*. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 7:1–7:21. <https://doi.org/10.4230/LIPIcs.ICDT.2022.7>
- [4] Noga Alon. 1988. The linear arboricity of graphs. *Israel Journal of Mathematics* 62, 3 (1988), 311–325.
- [5] Daichi Amagata. 2023. Diversity maximization in the presence of outliers. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 37. 12338–12345.
- [6] Daichi Amagata and Takahiro Hara. 2016. Diversified set monitoring over distributed data streams. In *Proceedings of the 10th ACM International Conference on Distributed and Event-Based Systems (Irvine, California) (DEBS '16)*. Association for Computing Machinery, New York, NY, USA, 1–12. <https://doi.org/10.1145/2933267.2933298>
- [7] Aditya Bhaskara, Mehrdad Ghadiri, Vahab Mirrokni, and Ola Svensson. 2016. Linear relaxations for finding diverse elements in metric spaces. In *Proceedings of the 30th International Conference on Neural Information Processing Systems (Barcelona, Spain) (NIPS'16)*. Curran Associates Inc., Red Hook, NY, USA, 4105–4113.
- [8] Sayan Bhattacharya, Sreenivas Gollapudi, and Kamesh Munagala. 2011. Consideration set generation in commerce search. In *Proceedings of the 20th international conference on World wide web*. 317–326.
- [9] Allan Borodin, Aadhar Jain, Hyun Chul Lee, and Yuli Ye. 2017. Max-sum diversification, monotone submodular functions, and dynamic updates. *ACM Transactions on Algorithms (TALG)* 13, 3 (2017), 1–25.
- [10] Grigori Calinescu, Howard Karloff, and Yuval Rabani. 2005. Approximation algorithms for the 0-extension problem. *SIAM J. Comput.* 34, 2 (2005), 358–372.
- [11] Pablo Castells, Neil Hurley, and Saul Vargas. 2021. Novelty and diversity in recommender systems. In *Recommender systems handbook*. Springer, 603–646.
- [12] Matteo Ceccarello, Andrea Pietracaprina, and Geppino Pucci. 2018. Fast Coreset-based Diversity Maximization under Matroid Constraints. In *Proceedings of the Eleventh ACM International Conference on Web Search and Data Mining (Marina Del Rey, CA, USA) (WSDM '18)*. Association for Computing Machinery, New York, NY, USA, 81–89. <https://doi.org/10.1145/3159652.3159719>
- [13] Elisa Celis, Vijay Keswani, Damian Straszak, Amit Deshpande, Tarun Kathuria, and Nisheeth Vishnoi. 2018. Fair and diverse DPP-based data summarization. In *International conference on machine learning*. PMLR, 716–725.
- [14] L Elisa Celis, Damian Straszak, and Nisheeth K Vishnoi. 2017. Ranking with fairness constraints. *arXiv preprint arXiv:1704.06840* (2017).
- [15] Alfonso Cevallos, Friedrich Eisenbrand, and Rico Zenklusen. 2017. Local search for max-sum diversification. In *Proceedings of the Twenty-Eighth Annual ACM-SIAM Symposium on Discrete Algorithms*. SIAM, 130–142.
- [16] Barun Chandra and Magnús M Halldórsson. 2001. Approximation Algorithms for Dispersion Problems. *Journal of Algorithms* 38, 2 (2001), 438–465. <https://doi.org/10.1006/jagm.2000.1145>
- [17] Marina Drosou and Evaggelia Pitoura. 2013. Diverse set selection over dynamic data. *IEEE Transactions on Knowledge and Data Engineering* 26, 5 (2013), 1102–1116.
- [18] Marwa El Halabi, Slobodan Mitrović, Ashkan Norouzi-Fard, Jakub Tardos, and Jakub M Tarnawski. 2020. Fairness in streaming submodular maximization: Algorithms and hardness. *Advances in Neural Information Processing Systems* 33 (2020), 13609–13622.
- [19] Erhan Erkut. 1990. The discrete p-dispersion problem. *European Journal of Operational Research* 46, 1 (1990), 48–60.
- [20] Jittat Fakcharoenphol, Satish Rao, and Kunal Talwar. 2004. A tight bound on approximating arbitrary metrics by tree metrics. *J. Comput. System Sci.* 69, 3 (2004), 485–497. <https://doi.org/10.1016/j.jcss.2004.04.011> Special Issue on STOC 2003.
- [21] Michael R Fellows. 1990. Transversals of vertex partitions in graphs. *SIAM Journal on Discrete Mathematics* 3, 2 (1990), 206–215.
- [22] Teófilo F Gonzalez. 1985. Clustering to minimize the maximum intercluster distance. *Theoretical computer science* 38 (1985), 293–306.
- [23] Sarel Har-Peled and Saladi Rahul. 2018. Two (known) results about graphs with no short odd cycles. *arXiv preprint arXiv:1810.01832* (2018).
- [24] Penny Haxell. 2011. On forming committees. *The American Mathematical Monthly* 118, 9 (2011), 777–788.
- [25] Marius Kaminskis and Derek Bridge. 2016. Diversity, serendipity, novelty, and coverage: a survey and empirical analysis of beyond-accuracy objectives in recommender systems. *ACM Transactions on Interactive Intelligent Systems (TiiS)* 7, 1 (2016), 1–42.
- [26] Matthäus Kleindessner, Pranjal Awasthi, and Jamie Morgenstern. 2019. Fair k-center clustering for data summarization. In *International Conference on Machine Learning*. PMLR, 3448–3457.
- [27] Michael J Kuby. 1987. Programming models for facility dispersion: The p-dispersion and maximum dispersion problems. *Geographical Analysis* 19, 4 (1987), 315–329.
- [28] Iiro Kumpulainen, Florian Adriaens, and Nikolaj Tatti. 2024. Max-Min Diversification with Asymmetric Distances. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 1440–1450.
- [29] Yash Kurkure, Miles Shamo, Joseph Wiseman, Sainyam Galhotra, and Stavros Sintos. 2024. Faster Algorithms for Fair Max-Min Diversification in Rd. *Proceedings of the ACM on Management of Data* 2, 3 (2024), 1–26.
- [30] Po-Shen Loh and Benny Sudakov. 2007. Independent transversals in locally sparse graphs. *Journal of Combinatorial Theory, Series B* 97, 6 (2007), 904–918. <https://doi.org/10.1016/j.jctb.2007.02.003>
- [31] Antonis Matakos, Sijing Tu, and Aristides Gionis. 2020. Tell me something my friends do not know: Diversity maximization in social networks. *Knowledge and Information Systems* 62 (2020), 3697–3726.
- [32] Manor Mendel and Assaf Naor. 2006. Ramsey partitions and proximity data structures. In *Proceedings of the 47th Annual IEEE Symposium on Foundations of Computer Science (FOCS '06)*. IEEE Computer Society, USA, 109–118. <https://doi.org/10.1109/FOCS.2006.65>
- [33] Zafeiria Moumoulidou, Andrew McGregor, and Alexandra Meliou. 2020. Diverse Data Selection under Fairness Constraints. In *International Conference on Database Theory*. <https://api.semanticscholar.org/CorpusID:224703414>
- [34] Lu Qin, Jeffrey Xu Yu, and Lijun Chang. 2012. Diversifying Top-k Results. *Proc. VLDB Endow.* 5, 11 (jul 2012), 1124–1135. <https://doi.org/10.14778/2350229.2350233>
- [35] Filip Radlinski and Susan Dumais. 2006. Improving personalized web search using result diversification. In *Proceedings of the 29th annual international ACM SIGIR conference on Research and development in information retrieval*. 691–692.
- [36] Sekharipuram S Ravi, Daniel J Rosenkrantz, and Giri Kumar Tayi. 1994. Heuristic and special case algorithms for dispersion problems. *Operations Research* 42, 2 (1994), 299–310.
- [37] Arie Tamir. 1991. Obnoxious facility location on graphs. *SIAM Journal on Discrete Mathematics* 4, 4 (1991), 550–567.
- [38] Jan Van Den Brand, Li Chen, Rasmus Kyng, Yang P Liu, Richard Peng, Maximilian Probst Gutenberg, Sushant Sachdeva, and Aaron Sidford. 2023. A deterministic almost-linear time algorithm for minimum-cost flow. In *2023 IEEE 64th Annual Symposium on Foundations of Computer Science (FOCS)*. IEEE, 503–514.
- [39] Da-Wei Wang and Yue-Sun Kuo. 1988. A study on two geometric location problems. *Information processing letters* 28, 6 (1988), 281–286.
- [40] Yanhao Wang, Francesco Fabbri, and Michael Mathioudakis. 2022. Streaming algorithms for diversity maximization with fairness constraints. In *2022 IEEE 38th International Conference on Data Engineering (ICDE)*. IEEE, 41–53.
- [41] Yanhao Wang, Francesco Fabbri, Michael Mathioudakis, and Jia Li. 2023. Fair max-min diversity maximization in streaming and sliding-window models. *Entropy* 25, 7 (2023), 1066.
- [42] Yanhao Wang, Michael Mathioudakis, Jia Li, and Francesco Fabbri. 2023. Max-min diversification with fairness constraints: Exact and approximation algorithms. In *Proceedings of the 2023 SIAM International Conference on Data Mining (SDM)*. SIAM, 91–99.
- [43] Dong Xin, Hong Cheng, Xifeng Yan, and Jiawei Han. 2006. Extracting redundancy-aware top-k patterns. In *Proceedings of the 12th ACM SIGKDD international conference on Knowledge discovery and data mining*. 444–453.
- [44] Sepehr Zadeh, Mehrdad Ghadiri, Vahab Mirrokni, and Morteza Zadimoghaddam. 2017. Scalable feature selection via distributed diversity maximization. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 31.
- [45] Kaiping Zheng, Hongzhi Wang, Zhixin Qi, Jianzhong Li, and Hong Gao. 2017. A survey of query result diversification. *Knowledge and Information Systems* 51 (2017), 1–36.